

ECE 165 Project

Elisey Ovchinnikov

A16864598

Andrew Gooboian

A17685394

Roberto Reyes

A15988582

In this project, our goal was to design a custom 8-bit adder with improved speed compared to the 8-bit ripple carry adder that we synthesized in Lab 4. It is important that we are able to create fast and efficient adders because they serve as fundamental building blocks of components such as arithmetic logic units (ALUs), which are essential to a computer's ability to perform operations. By designing our own adder architecture, we are able to achieve lower delays than those produced by generic RTL synthesis, demonstrating why custom hardware design is important in high-performance computing systems.

One of the most common architectures for an 8-bit adder is the ripple carry adder (RCA), valued for its simplicity and compact design. However, this architecture is not optimal for speed because of its serial nature. Each bit within the RCA has to wait for the previous carry-out bit calculation in order to do its own calculation, leading to a very high propagation delay. When building for speed, we must have an architecture that allows bits to be calculated in parallel to reduce calculation delay, such as carry-lookahead or carry-skip adders.

We chose to utilize a carry-lookahead abridged with a carry-skip, which can be seen in Figure 3. This hybrid design allows us to get fast carry outputs locally so we don't rely on the propagation of earlier bits with the CLA and the carry-skip allows us to skip certain bits speeding up the adder process even more for certain calculations.

We were able to innovate by having four different blocks work in parallel for maximum efficiency. Two 3-bit CLA's (Figure 1) and two half adder designs (Figure 2). These two parts were blocked together and connected by the carry skip allowing the second CLA to not have to wait for bits to be propagated. This design is slower than a full 8-bit CLA design, but it makes up for that by requiring much less transistors and consuming much less power.

An unfortunate error that occurred during testing that took 2 hours to troubleshoot was a simple error with the test bench. Originally when the first two bits were added $1 + 1$ we should have gotten an output of 0 through our XOR

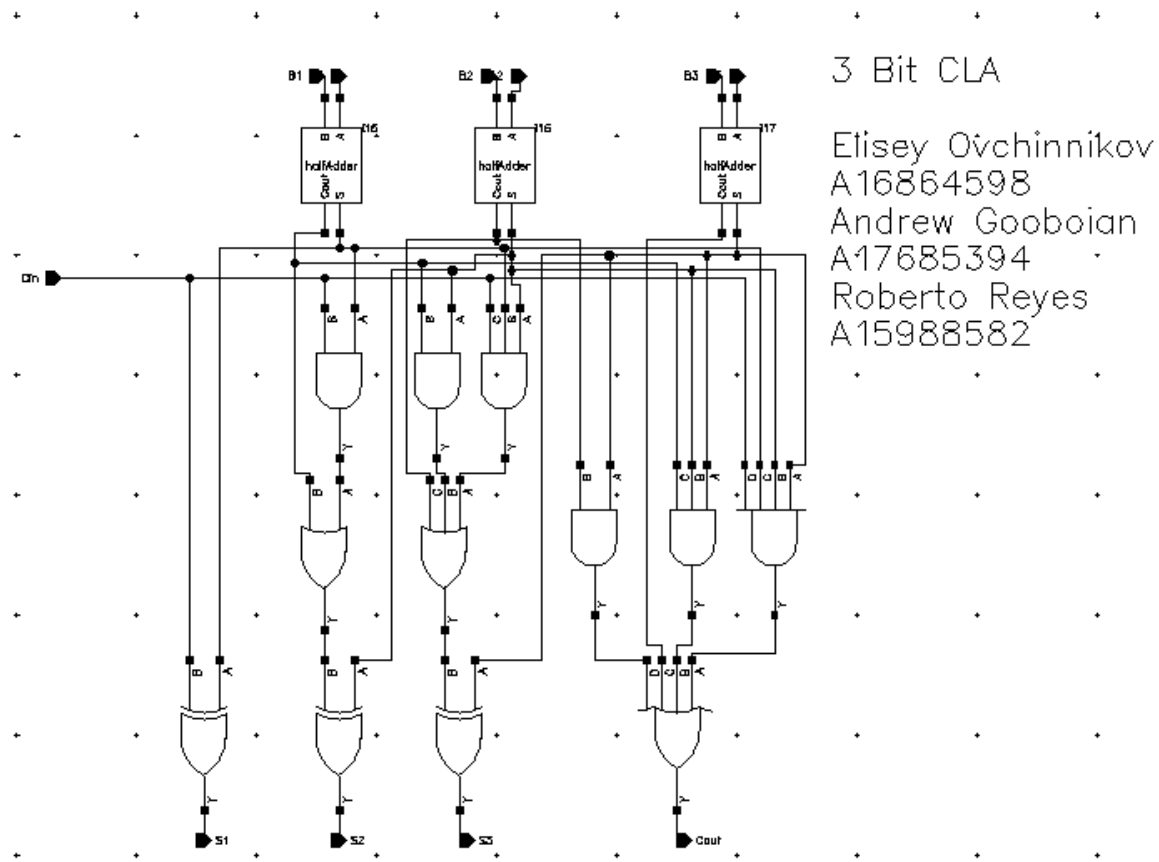
gate but for some reason no matter what we did, we kept getting 1 no matter what we changed. It turned out that our input bits in the test bench were reversed to what we had in our design. The test bench inputted and output values as $X<7:0>$ while our design had the backwards as $X<0:7>$. Once we noticed this flaw it was a quick fix that instantly then gave out the correct outputs.

For future iterations, we could further optimize transistor sizes and redundant transistors to decrease propagation delay. This was attempted during the design process and provided some advancements in reducing the power consumption (Figure 7), but ultimately didn't produce results with higher maximum frequencies than the original design with standard gsclib045 cells.

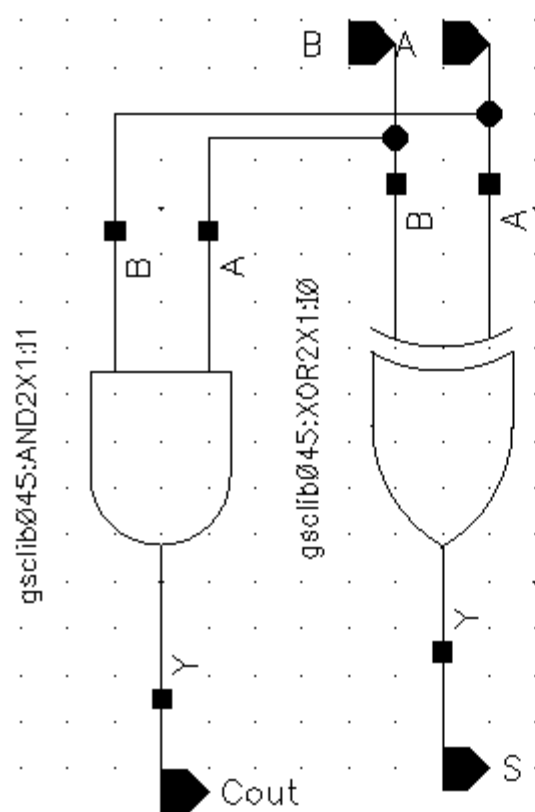
We split the project workload as evenly as possible with each person designing bits and pieces of the 8 bit adder. We came up with our 8-bit adder architecture as a group and then split off to do separate pieces for it. Roberto created the schematic and symbol for the half adder as seen in Figure 2. Andrew created the 3-bit CLA schematic and symbol which can be seen in Figure 1. Elisey took all the designs and symbols to create a final schematic and symbol of the 8-bit adder which can be seen in Figure 3. Finally, to test our adder, we all met together at office hours for the project and made sure that the adder produced the correct output at frequencies above 1.8GHz and tested to make sure it correctly added the two input values. This resulted in maximum propagation delay in the test bench which can be seen done in Figure 4. Then we came back the next day and were able to get all of our transient graphs done and calculate the performance metrics at the different frequencies that were required which can be seen done in the filled out table of Figure 5.

Acknowledgements:

Thank you to our teaching assistant, Theresa Pia Tovey, for helping us answer our questions and help us troubleshoot during office hours.



¹ Schematic of 3-bit carry-lookahead adder



Half Adder

Elisey Ovchinnikov

A16864598

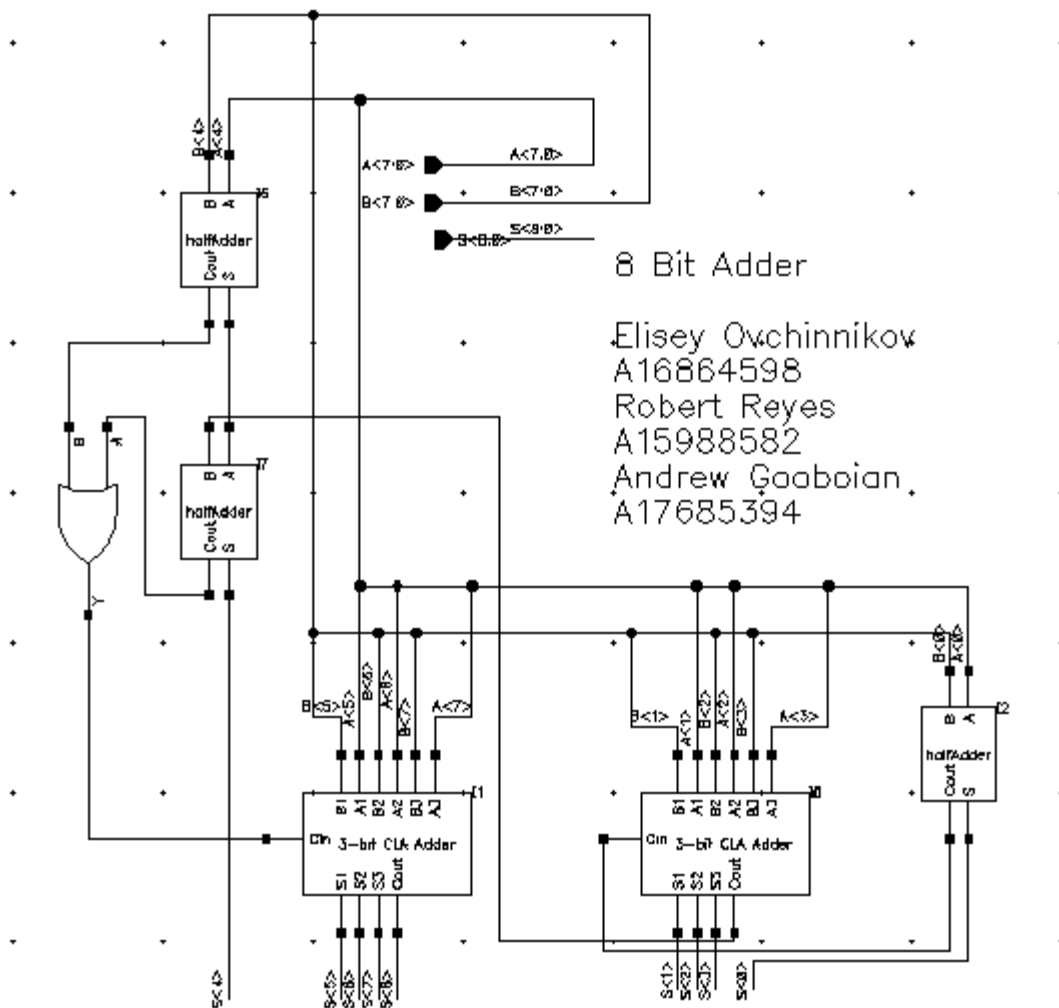
Andrew Gooboian

A17685394

Roberto Reyes

A15988582

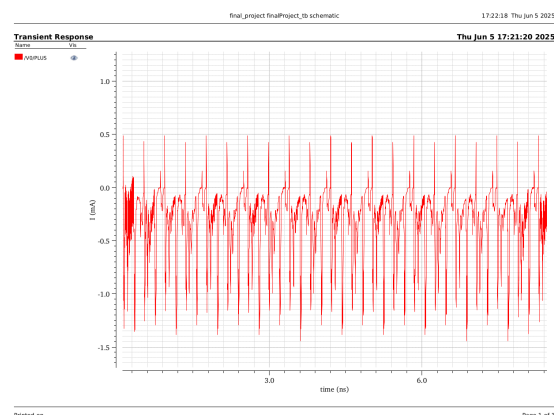
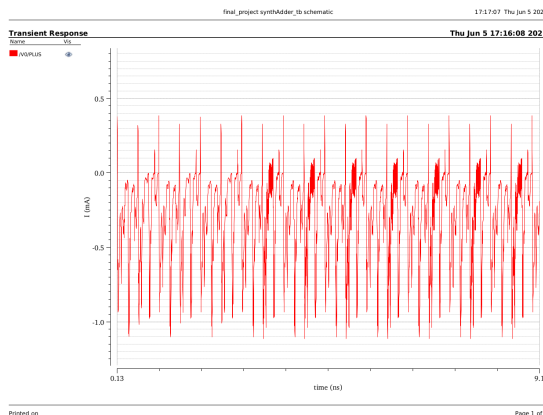
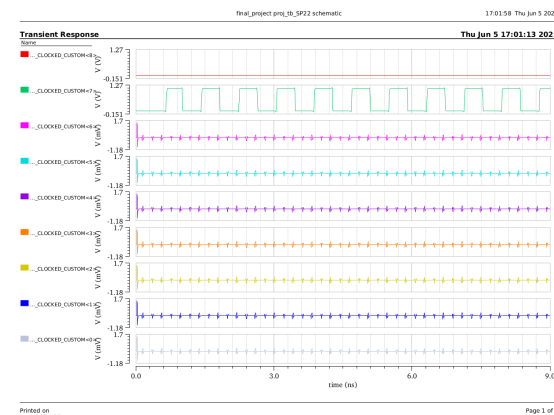
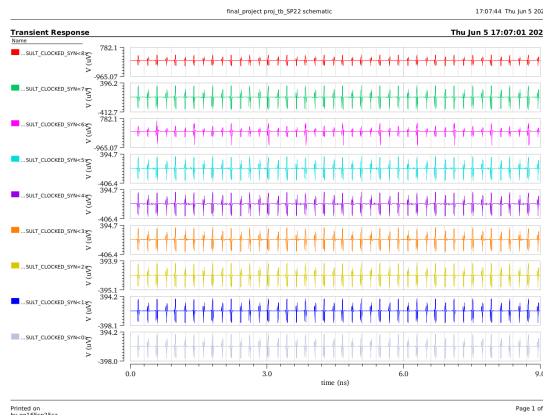
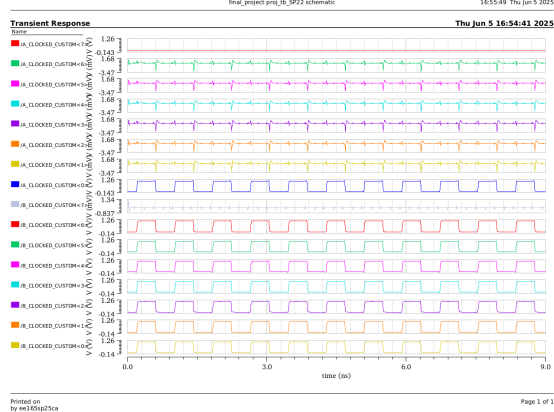
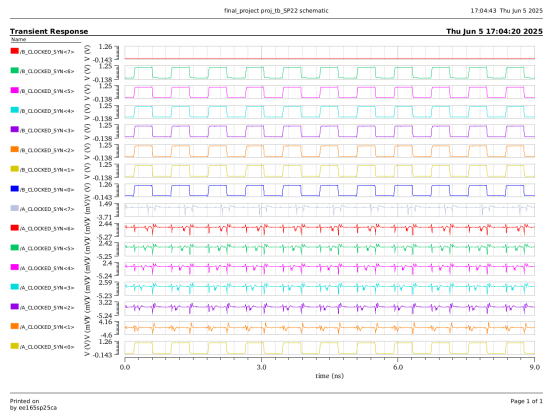
² Schematic of half-adder



3

³ Schematic of 8-bit adder

UCSD - ECE 165: DIGITAL INTEGRATED CIRCUIT DESIGN PROJECT



4

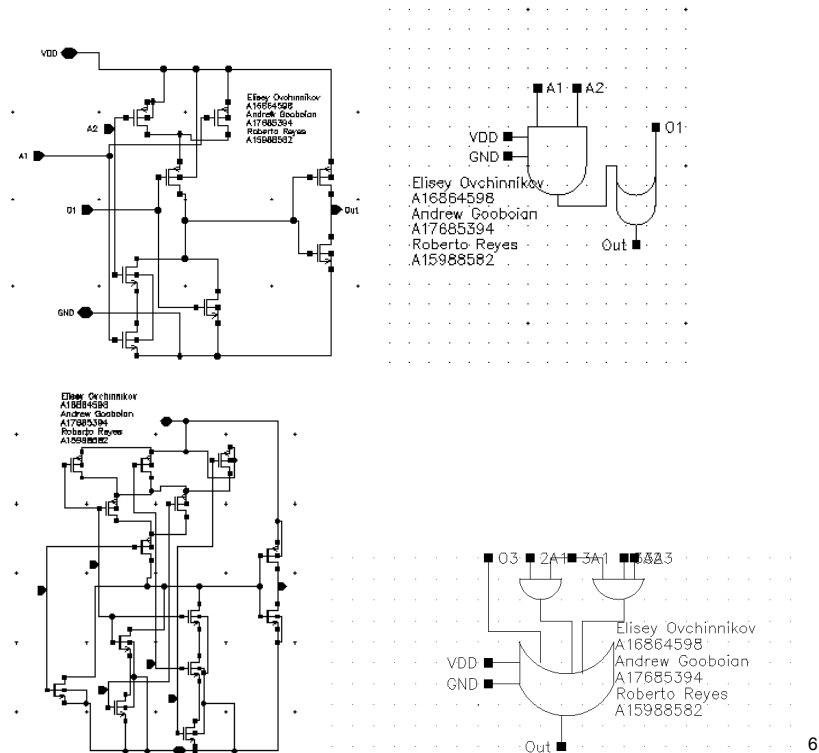
⁴ (Left side) Synthesized A,B, results, and current
(Right side) Custom A,B, results, and current

	Place+route schematic	Custom Design Schematic
Performance for VDD = 1.1V		
f_{max}	2.27GHz	2.45GHz
Power Consumption @ f_{max}	314.27uW	340.107uW
Energy per operation @ f_{max}	138.445fJ	138.819fJ
Performance for VDD = 1.1V, $f_{CLK} = 1$ GHz		
Power Consumption @ 1Ghz	136.528uW	137.975uW
Energy per operation @ 1GHz	136.528fJ	137.975fJ
Other Important Parameters		
Adder Architecture	Ripple Carry	Carry Look Ahead Abridged with Carry Skip
Critical Input Pair	A = 00000001 B = 01111111	A = 00000001 B = 01111111
Transistor types used	VTG	VTG

5

⁵ Table of results

Alternative circuits which gave lower power consumption, but were ultimately scrapped due to a lower max frequency:



⁶ Extra Designs